

OQQQ 开发手册

仓库结构、环境搭建、协作规范与前后端联调的统一工程基线

版本

v0.1 · Draft Baseline

日期

2026-10-07

适用对象

前端、后端、Agent 适配器与测试开发者

基线假设

TypeScript 单仓库；React 前端；Node.js 服务端

使用方式

本手册是开工基线，不代表 GitHub 仓库已经创建。仓库初始化后，应将本文件放入 `docs/`，并通过架构决策记录（ADR）确认或替换尚未锁定的技术选型。

先统一规则，再开始写代码

OQQQ 是一个 QQ 式多 Agent 协作客户端：消息、联系人、会话工作区、技能、连接器与第三方插件市场共用一套前端，后端通过适配器连接 Hermes、Claude Code、Codex、DeepSeek Harness 等不同引擎。

01 仓库结构

单仓库、目录职责、依赖边界

02 环境搭建

版本锁定、配置与初始化

03 本地运行

启动、调试、日志和健康检查

04 代码规范

TypeScript、组件、服务端与安全边界

05 Git 协作

提交、分支与合并规则

06 修复报告

每次代码修改的硬性记录要求

07 API 契约

变更流程、兼容窗口与版本化

08 前后端联调

Mock、WebSocket 与错误定位

09 测试与交付

四层测试、CI 与完成定义

10 常见问题

环境、端口、契约与工作区故障

工程原则

契约先行

页面和服务端并行开发时，先合并 OpenAPI / WebSocket 事件契约，再各自实现。

边界校验

外部模型、插件、文件系统和用户输入全部视为不可信输入，在入口校验。

可追溯

消息、工具调用和错误都携带关联 ID；每次代码变更都有修复报告。

技术栈假设：本手册推荐 Node.js 20 LTS、pnpm、React + TypeScript + Vite、Fastify + WebSocket。若前端团队已确定其他框架，先提交 ADR，再同步更新本手册和脚本。

单仓库让契约、类型与版本一起演进

推荐使用 pnpm workspace。应用只负责组装，复用能力放入 packages；跨端共享的类型和契约只能有一个来源。

```
oqqq/
├── apps/
│   ├── desktop/      # QQ 式桌面前端
│   └── server/        # REST、WebSocket、消息总线
├── packages/
│   ├── contracts/     # OpenAPI、事件 schema、生成类型
│   ├── agent-core/    # AgentAdapter 接口与注册表
│   ├── agent-adapters/ # hermes / claude-code / codex / dsh
│   ├── workspace-core/ # 会话工作区、文件索引与权限
│   ├── plugin-client/  # 第三方插件市场客户端
│   ├── ui/            # 共享组件与设计 token
│   └── config/         # eslint、tsconfig、vitest 共享配置
├── docs/
│   ├── architecture/  # 架构说明和 ADR
│   ├── api/           # 生成后的契约文档
│   └── fix-reports/    # 每次代码修改的修复报告
├── scripts/           # 开发、校验、生成代码脚本
├── pnpm-workspace.yaml
├── package.json
└── README.md
```

目录责任

目录	允许依赖	禁止事项
apps/desktop	ui、contracts、API 客户端	直接访问数据库或进程命令
apps/server	contracts、各领域 package	反向依赖桌面组件
contracts	schema 与纯类型	业务逻辑、网络请求
agent-adapters	agent-core、基础设施端口	泄漏具体 CLI 输出到 UI
workspace-core	路径、索引、权限抽象	跳过目录范围检查

依赖方向： apps → domain packages → contracts/config。出现循环依赖时，不用 barrel export 掩盖问题；提取共同接口或事件。

命名约定：目录与文件用 kebab-case；React 组件用 PascalCase；测试与源文件同目录，命名为 *.test.ts(x)。

用固定版本把“在我机器上能跑”变成可复现

基础环境

- **Node.js**: 20 LTS, 使用 `.nvmrc` 或 Volta 锁定。
- **包管理**: pnpm, 通过 `packageManager` 锁定精确版本。
- **Git**: 2.40 以上, 统一 LF; Windows 设置 `core.autocrlf=false`。
- **可选运行时**: 不同 Agent CLI 按适配器说明安装, 不能成为服务端启动的强依赖。

首次初始化

```
git clone <repository-url> oqqq
cd oqqq
corepack enable
pnpm install --frozen-lockfile
cp .env.example .env.local
pnpm bootstrap
pnpm check
```

环境变量分层

文件	用途	是否提交
<code>.env.example</code>	变量名、注释、非敏感默认值	是
<code>.env.local</code>	个人本地配置	否
<code>.env.test</code>	测试环境安全默认值	是
CI secret	发布密钥与平台令牌	否

```
# .env.example
OQQQ_SERVER_HOST=127.0.0.1
OQQQ_SERVER_PORT=3210
OQQQ_WEB_PORT=5173
OQQQ_DATA_DIR=./data
OQQQ_LOG_LEVEL=debug
# 真实模型 Key 只通过安全配置流注入，不进入仓库
```

安全要求: 不得把 API Key、Token、Cookie、私钥或本机绝对路径提交到 Git。日志必须对凭据、Authorization 头和敏感字段做脱敏。

前后端分别准备

前端

安装依赖后运行类型生成；确认七个一级入口可在 Mock 模式启动；浏览器控制台无未处理异常。

后端

初始化本地数据目录；启动健康检查；未安装任何 Agent CLI 时仍能返回可理解的能力状态。

一条命令启动，四个信号定位故障

```
# 启动前端、服务端和契约监听
pnpm dev

# 单独运行
pnpm --filter @oqqq/desktop dev
pnpm --filter @oqqq/server dev

# 全量校验
pnpm check      # typecheck + lint + test + contract check
pnpm test:e2e    # 端到端主流程
```

默认本地地址

服务	地址	用途
Desktop Web	127.0.0.1:5173	页面调试
REST API	127.0.0.1:3210/api	资源读写
WebSocket	127.0.0.1:3210/ws	消息与流式事件
Health	127.0.0.1:3210/health	启动探针

四个调试信号

1. requestId

每次 REST 请求由服务端生成；前端错误卡片可复制，日志可直接检索。

2. correlationId

一条用户消息、Agent 流式回复和工具调用共用，跨进程串联。

3. sessionId

标识单聊或群聊的运行会话，不把 Agent 进程 ID 暴露给 UI。

4. adapter trace

记录适配器状态机和退出码；输出脱敏后进入结构化日志。

调试顺序：先检查 `/health`，再检查 WebSocket 是否连接，然后用 `requestId` 查服务端日志，最后进入具体 Agent 适配器。不要先在 UI 猜原因。

日志最低字段

```
{ level, time, service, event,
  requestId?, correlationId?, sessionId?, agentId?,
  durationMs?, errorCode?, message }
```

UI 只认识统一能力，不认识具体 Agent CLI

不同引擎的安装、输入格式、工具事件与错误各不相同。适配器层负责归一化，消息总线负责路由，UI 只消费稳定契约。



AgentAdapter 最小接口

```

interface AgentAdapter {
  readonly type: AgentType;
  detect(): Promise<InstallState>;
  install?(input: InstallInput): Promise<InstallResult>;
  start(context: AgentContext): Promise<AgentSession>;
  send(sessionId: string, message: AgentInput): AsyncIterable<AgentEvent>;
  stop(sessionId: string): Promise<void>;
  listCapabilities(): Promise<Capability[]>;
}
  
```

不可跨越的边界

- **联系人 ≠ 进程**：一个已添加 Agent 是持久配置；一次聊天运行是短生命周期会话。
- **会话 ≠ 工作区**：会话记录可以归档，工作区路径必须显式保存、校验并可迁移。
- **插件 ≠ 技能 ≠ 连接器**：三者 UI 是独立一级功能；运行时通过能力清单绑定到 Agent。
- **消息事件 ≠ 原始 stdout**：适配器必须把流式文本、工具调用、状态与错误转换为统一事件。

进程控制：启动外部 CLI 必须限制工作目录、参数、环境变量和超时；停止时终止整个进程树并确认退出，避免孤儿进程。

严格类型、显式边界、可预测错误

TypeScript

- 开启 `strict`、`noUncheckedIndexedAccess`。
- 不使用裸 `any`；外部输入先是 `unknown`，再经 `schema` 解析。
- 公开函数写返回类型；跨包接口避免导出实现类。
- 时间统一传 ISO 8601 UTC，界面层再本地化。

React

- 组件只处理展示和交互；数据请求放 `query/service` 层。
- 状态按 `server`、`session`、`view` 三类分开，禁止一个全局大 `store`。
- 流式消息按 `messageId` 增量更新，避免整列表重渲染。

服务端

- 路由只做鉴权、校验与映射；业务规则进入 `domain service`。
- 错误使用稳定 `code`，不让客户端解析英文 `message`。
- 文件路径先规范化，再验证位于授权工作区内。
- 外部 Agent 事件必须能取消、超时并清理资源。

命名与注释

- 变量说明“是什么”，函数说明“做什么”；布尔值以 `is/has/can/should` 开头。
- 注释解释原因、约束或取舍，不复述代码。
- 删除注释掉的旧代码；历史由 Git 保存。

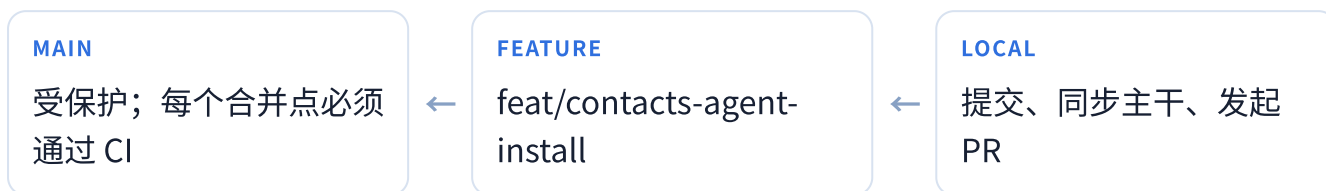
统一错误体

```
{
  "error": {
    "code": "AGENT_NOT_INSTALLED",
    "message": "Agent 尚未安装",
    "requestId": "req...",
    "details": { "agentType": "codex" },
    "retryable": false
  }
}
```

错误类别	示例前缀	UI 行为
输入无效	<code>VALIDATION_</code>	字段旁提示
Agent 状态	<code>AGENT_</code>	引导安装或重启
工作区	<code>WORKSPACE_</code>	显示路径与修复动作
外部服务	<code>UPSTREAM_</code>	展示可重试状态

短分支、小提交、主干始终可运行

分支策略



- **main**: 唯一长期分支，不直接推送；发布用 tag 标记。
- **feat/***: 新功能；**fix/***: 缺陷修复；**docs/***: 纯文档；**chore/***: 构建与依赖。
- 分支应在 1-3 天内合并；超过一周必须拆分或说明原因。
- 合并前 rebase 最新 main；推荐 squash merge，PR 标题作为最终提交标题。

Conventional Commits

<type>(<scope>): <summary>

```

feat(contacts): add agent installer panel
fix(ws): preserve message order after reconnect
docs(api): document workspace update event
refactor(adapter): normalize process exit handling
test(contract): cover agent capability schema
  
```

好提交

一次只表达一个意图；代码、测试、契约和修复报告一起提交；标题说明结果。

拒绝提交

“update” “fix bug” 等无信息标题；混入格式化全仓文件；提交生成缓存或真实凭据。

Pull Request 最低内容

- ☐ 说明问题、方案和用户可见变化。
- ☐ 列出验证命令、关键截图或事件样例。
- ☐ 链接 API 契约 / ADR / issue；标注破坏性变更。
- ☐ 链接本次 docs/fix-reports/ 报告。

审查重点：先看行为、权限、契约和失败路径，再看风格。自动格式化可由 CI 处理，不能替代架构审查。

每次代码修改都必须更新修复报告

硬规则：只要修改了源代码、配置、构建脚本、依赖或测试，就必须在 docs/fix-reports/ 新增或更新对应报告，文件名必须带执行修改的 AI 名称后缀。没有报告，修改不算完成。

命名格式

```
docs/fix-reports/YYYY-MM-DD-<topic>-<ai-name>.md

# 示例
docs/fix-reports/2026-10-07-agent-install-flow-muse.md
docs/fix-reports/2026-10-07-ws-reconnect-claude.md
```

报告模板

```
# <变更标题>

- 日期: YYYY-MM-DD
- 执行者: <AI 名称或开发者>
- 分支 / 提交: <branch / commit>
- 影响范围: <packages / apps>

## 问题
描述可复现症状、触发条件与影响。

## 根因
说明实际根因；不能确认时写“待确认”和现有证据。

## 修改
逐项列出文件与行为变化。

## 验证
列出实际执行的命令、结果和关键边界用例。

## 风险与回滚
说明剩余风险、兼容性与回滚方式。
```

何时新增，何时更新

情形	动作	要求
独立新问题	新增报告	一问题一报告
同一问题迭代	更新原报告	保留变更时间线
多人 / 多 AI 接力	各自报告	各自名称后缀
纯排版文档	可不建报告	不影响代码行为

CI 建议：当 PR 改动 apps/、packages/、构建配置或 lockfile 时，校验本次 diff 至少包含一个合法命名的 docs/fix-reports/*-<name>.md。

契约先合并，代码再并行

REST 使用 OpenAPI，WebSocket 使用独立事件 schema。前后端不得手写两套同名类型；生成类型是唯一消费入口。



WebSocket 事件信封

```

{
  "type": "message.delta",
  "eventId": "evt_...",
  "occurredAt": "2026-10-07T12:00:00.000Z",
  "conversationId": "conv_...",
  "correlationId": "corr_...",
  "payload": { "messageId": "msg_...", "delta": "..." }
}
  
```

兼容性规则

变更	兼容性	处理
新增可选字段 / 事件	通常兼容	增加测试与示例
字段改名 / 删除	破坏性	先新增、标记废弃、跨版本删除
枚举新增值	可能破坏	客户端必须有 unknown 分支
语义改变	破坏性	新字段或新版本，不复用旧名

契约变更 PR 必须包含

- ☐ 更新 schema、成功示例、错误示例和事件顺序说明。
- ☐ 重新生成类型并提交生成物；CI 校验无未提交差异。
- ☐ 列出旧客户端 / 旧服务端组合的行为。
- ☐ 若破坏兼容，给出迁移步骤、灰度窗口和删除版本。
- ☐ 同步更新前端设计文档与开发手册相关章节。

版本策略：v0 阶段也不能“随便改”。每次破坏性变更在变更记录中标明，并保证同一 PR 内前后端可独立启动。

Mock 验布局，真实服务验行为

两种联调模式

Contract Mock

由契约示例生成固定响应和事件序列。前端可独立完成消息、联系人、工作区、技能、连接器、插件市场与设置页面。

Real Server

连接本地服务端，验证安装状态、进程生命周期、文件权限、断线重连、流式消息和错误恢复。

联调主路径

1. 联系人页点击“添加 Agent”，读取可安装 Agent 列表。
2. 安装或检测完成后，好友列表出现持久 Agent 配置。
3. 创建单聊，选择独立工作区，发送消息并接收增量事件。
4. 创建群聊，添加多个 Agent；@Agent 只路由给目标成员。
5. 打开总工作区，能定位到每个单聊 / 群聊的目录与文件。
6. 绑定技能、连接器和插件，能力变化实时同步到详情页。

WebSocket 最低行为

场景	客户端	服务端
连接成功	发送 resume token	回 welcome 与游标
断线	指数退避，显示状态	保留有限事件窗口
重连	携带 lastEventId	补发或要求全量刷新
重复事件	按 eventId 去重	事件 ID 全局唯一
取消回复	发 cancel 命令	停止适配器并回终止事件

联调证据：问题单必须包含页面、操作步骤、requestId / correlationId、期望与实际结果。只有截图而没有关联 ID 的问题，不足以定位后端链路。

工作区、能力与 Agent 生命周期各自独立

会话工作区

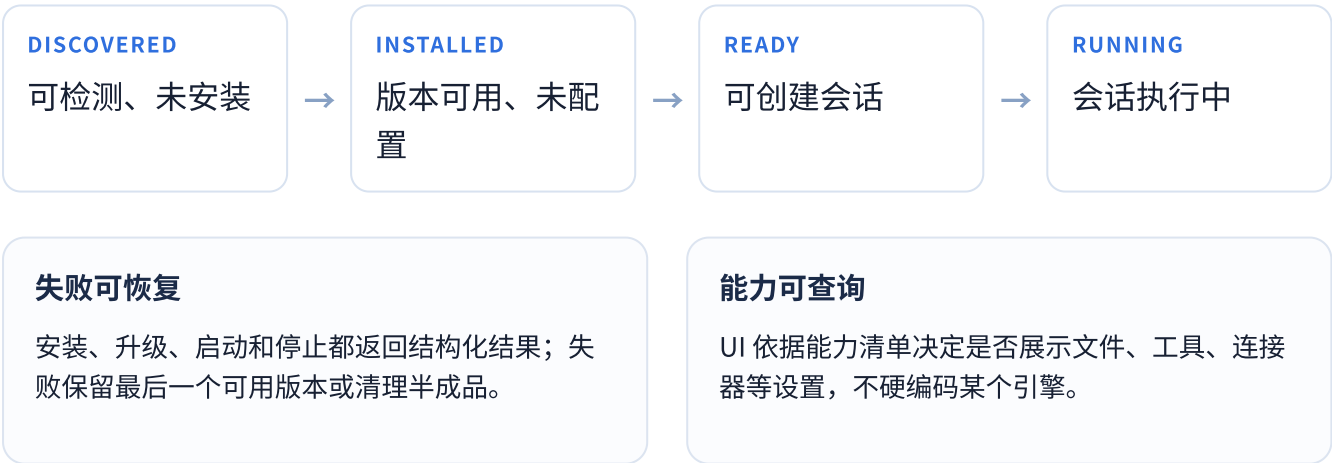
- 每个单聊和群聊都保存独立 `workspaceId`，指向经授权的本地目录。
- 左侧“工作区”是总视图，聚合所有会话，但不改变各会话的目录边界。
- 路径变更必须检查存在性、读写权限、符号链接逃逸和冲突；失败时保留旧配置。
- Agent 工具调用默认只能访问当前会话工作区，跨工作区必须显式授权。

三类能力

类型	定位	绑定方式
技能	可复用工作流 / 指令能力	绑定 Agent 或会话
连接器	外部服务与账户访问	先授权，再绑定
插件	dsh 第三方扩展包	安装、校验、启用

第三方插件市场的数据源由产品配置指定。安装前展示来源、版本、权限、兼容范围和校验结果；安装失败不得留下“已安装”的假状态。

Agent 状态机

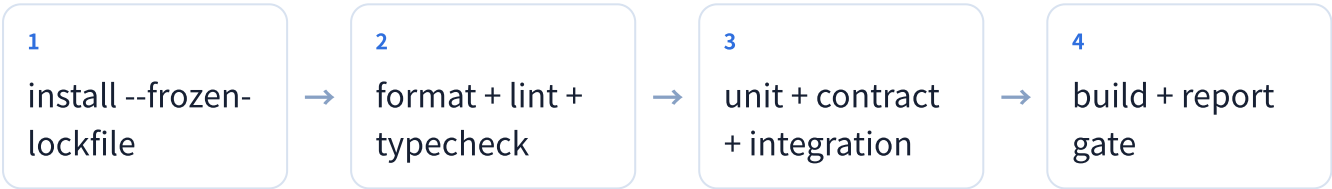


首版范围建议：先实现“检测已安装 CLI + 启动会话 + 流式消息 + 停止”，把自动下载安装作为单独里程碑，避免把系统包管理风险混入聊天主链路。

四层测试共同证明主流程可交付

层级	覆盖内容	运行时机
单元	路由、状态机、路径校验、事件归一化	本地 watch / 每次 PR
契约	OpenAPI、事件 schema、生成类型一致	每次 PR
集成	数据库 / 文件系统 / 假 Agent 进程	每次 PR
端到端	添加 Agent、单聊、群聊、工作区、重连	主干 / 发布候选

CI 建议流水线



关键边界用例

- ☐ Agent 未安装、版本不兼容、启动超时。
 - ☐ 进程异常退出、部分输出、取消与重复停止。
 - ☐ WebSocket 断线、补发、重复事件和乱序。
 - ☐ 群聊 @ 不存在成员或多个同名成员。
- ☐ 工作区不存在、只读、符号链接越界。
 - ☐ 插件包损坏、权限超范围、升级失败。
 - ☐ 超长消息、非法 schema、未知枚举值。
 - ☐ 前端刷新后会话、草稿与运行状态恢复。

完成定义 (Definition of Done)

- ☐ 需求验收条件全部通过，失败路径有可操作提示。
- ☐ 类型、lint、测试、构建和契约检查通过。
- ☐ 新增或更新测试；无静默跳过用例。
- ☐ 相关 API 文档、ADR 和用户可见文案已同步。
- ☐ 已更新带 AI 名称后缀的修复报告。
- ☐ 无凭据、调试垃圾、绝对路径或未说明的破坏性变更。

常见问题按信号排查，不靠猜

现象	先检查	处理
pnpm install 失败	Node / pnpm 版本、lockfile	切到锁定版本；禁止删除 lockfile 碰运气
前端空白	控制台、API base URL、生成类型	先用 Mock 启动，再检查真实服务
服务端启动失败	端口、.env.local、数据目录	查看 health 与结构化启动日志
Agent 显示未安装	detect 结果、PATH、版本输出	在服务端同一环境执行检测
消息不流式	WS 状态、correlationId、事件顺序	检查代理缓冲与适配器归一化
重复消息	eventId 去重、重连游标	修正客户端幂等和补发边界
群聊 @ 错人	成员 ID 而非显示名	输入时锁定 memberId，展示名仅显示
工作区无法写入	实际路径、权限、越界检查	返回明确错误，不自动放宽权限
插件安装后不可用	兼容范围、启用状态、能力刷新	回滚安装并保留诊断信息
前后端字段不一致	schema 与生成物 diff	重新生成；禁止手改生成类型

提交问题前的最小诊断包

- ☐ 当前分支和提交 SHA。
- ☐ Node、pnpm、操作系统版本。
- ☐ 复现步骤与期望 / 实际结果。
- ☐ requestId / correlationId。
- ☐ 脱敏后的相关日志片段。
- ☐ Agent 类型、版本与检测状态。
- ☐ 是否使用 Mock 或真实服务。
- ☐ 最近一次契约生成是否干净。

禁止做法：不要为“修好”问题而关闭校验、跳过权限、吞掉错误、降低 TypeScript 严格度、提交真实凭据，或把失败状态伪装成成功。

仍无法定位时：缩小到最小复现；替换真实 Agent 为假适配器；保留同一 correlationId 对比事件序列。问题落在哪一层，就由哪一层修复，不在相邻层做补丁遮盖。

新开发者第一天清单

完成以下清单后再领取功能任务。任何一步失败都应先修复工程基线，而不是绕过。

环境

- ☐ 安装仓库锁定的 Node 与 pnpm 版本。
- ☐ 使用 frozen lockfile 安装成功。
- ☐ 复制本地配置且未放入任何真实凭据。
- ☐ `pnpm check` 全部通过。

运行

- ☐ 前端 Mock 模式可打开七个一级入口。
- ☐ 本地服务端 health 返回健康。
- ☐ WebSocket 可连接、断线并恢复。
- ☐ 假 Agent 可完成一次流式回复。

协作

- ☐ 阅读最新 ADR 与 API 契约。
- ☐ 从 main 创建短分支。
- ☐ 按 Conventional Commits 提交。
- ☐ PR 关联修复报告和验证证据。

交付

- ☐ 主流程与失败路径均有测试。
- ☐ 契约与生成类型无差异。
- ☐ 工作区、日志和凭据通过安全检查。
- ☐ 完成定义逐项确认。

首批建议里程碑

M0 • FOUNDATION

仓库、契约、
CI、Mock、修
复报告门禁



M1 • CHAT

单聊、群聊、@
路由、流式消息



M2 • AGENTS

适配器、检测、
启动、停止、详
情设置



M3 • ECOSYSTEM

工作区、技能、
连接器、插件市
场

基线所有者：仓库维护者。任何人都可以提议改变本手册，但必须通过 ADR 说明动机、取舍、迁移方式和生效版本。

下一步：GitHub 仓库创建后，先落地 M0；将本手册放入 `docs/OQQQ-开发手册.pdf`，同时维护可审阅的 Markdown 或源文档版本。